ADR-001: DevEx Golden Path Ecosystem

> **Status:** Final · **Author:** Jorge Flores · **Date:** 2026-05-26
>
> **Decision.** Build a polyglot Golden Path as two independently distributable packages — `devex-cli` (Python, `uv tool install`) and `@devex/framework` (TypeScript, `pnpm add`) — homologating the lifecycle across 10+ teams without putting the platform team in the critical path.
>
> **Context.** Org scaling from a handful of teams to 10+. DORA metrics are not comparable across teams. SOC 2 audit is painful because each team owns CI/CD differently. The current platform team is a bottleneck for every CI/CD change.

1. Architecture overview

> The full architecture diagram lives in the project README.

Key properties. Two products with separate versioning, install commands, and consumers — coupled only by shared schemas (DORA event, Work ID regex, audit log) defined once in the monorepo and consumed by both. A single Git tag releases CLI + Framework atomically; cross-cutting changes are one PR. The `StackProfile` discriminated union (`python-lambda-api`, `go-lambda-api`, …) is the only place a language is named — adding Go means one new profile plus stub implementations, **not** a framework fork. DORA and audit events use a standardized schema; the framework defines the contract, the org's existing pipelines define storage.

2. Homologation

Adoption is engineered, not requested.

Lever	Mechanism
Lower the cost	`devex init` scaffolds the entire Golden Path in one command (workflows, lint configs, hooks, branch protection) and auto-detects `repo-url` from `git remote` plus `service-name` from the working directory — zero required flags. Time-to-first-green-PR drops from days to under 10 minutes.
Self-enforce	Four checkpoints fire before the platform team is involved: (1) IDE lint/format/type-check on save; (2) pre-push hook calls `devex validate` plus optional `devex check` for test/lint; (3) generated PR pipeline re-runs validation, contract checks, and `cdk synth` with FinOps-tag enforcement; (4) branch protection blocks merges without required status checks. A repo that doesn't adopt simply cannot merge.
Migrate safely	`devex init --migrate` runs inspect-only by default for existing repos. `--apply` writes incremental PRs the team reviews. Eventual convergence, never big-bang. Reruns are idempotent (UNIX exit codes: `0` = ok or no-op, `1` = partial state, `2` = misuse).
Friction-free DX	Work IDs auto-inject into commit messages via `prepare-commit-msg` (developer never types `KAN-123:` again). Hooks bake the absolute `devex` path at install time so they survive Git's sanitized `PATH`. The pre-push hook is bypassable with `--no-verify` — the PR pipeline re-validates and cannot be bypassed.

Anti-patterns rejected: voluntary adoption (fails at scale), big-bang migration (kills trust), mandatory training (if the CLI needs a class, we've failed at DX), hook-as-moat (treat hooks as gifts to the developer; let branch protection be the moat).

3. Scalability — keeping the platform team out of the critical path

Three mechanisms make the platform team scale sub-linearly with team count.

- **Extension over modification.** Every Construct and workflow factory accepts override hooks (`smallTestsJob(profile).addStep(customStep)`, `props.lambdaOverrides`). Teams customize without forking. The `StackProfile` discriminated union with `assertNever` exhaustiveness means adding a language is additive, never invasive.

- **Inner-source.** Any team can PR the monorepo. The framework's own pipeline is the gate; review goes to a rotating maintainer (1 platform + 1 IC from consumer teams). Breaking changes require a 5-day RFC; additive changes don't. Adding a new language = one file under `src/profiles/`, no core changes.
- **Platform team's actual job:** maintain contracts, curate inner-source contributions, run the upgrade path (semver, deprecation). **Not:** write per-team CI/CD, debug per-team failures, gatekeep custom infra.

4. Shift-Left strategy

Defects detected where they are cheapest. Four layers, ordered left-to-right; each layer's bypass is re-caught at the next:

1. **IDE / Local** — lint, format, type-check on save, configured by `devex init` (no new tools, just enforced configs).
2. **Pre-push hook** — `devex validate` checks Work ID in branch/commits/PR title; `devex check` runs lint then tests (fail-fast). Bypassable with `--no-verify` — re-validated at L3.
3. **PR Pipeline** — full lint, unit + property-based tests, contract validation against `openapi.yaml`, `cdk synth` with FinOps tag enforcement via Aspects (priority `READONLY`), DORA + audit event emission. Required status check on `main`.
4. **Integration Pipeline (designed, deferred)** — promotion across Sandbox → Staging → Prod. Multi-env is already structured (Pattern A: one `Stack` instance per env in `bin/`); the PoC ships the PR pipeline only.

Auditability is a dividend, not an afterthought. Every stage emits one structured JSON event with shared keys (`work_id`, `team`, `repo`, `stage`, `status`, `actor`, `timestamp`, `git_sha`, `framework_version`). The same event stream feeds DORA dashboards and SOC 2 audit — one primitive, two consumers.

5. Distribution and contracts

Both products install from Git directly (`uv tool install` for the CLI, `pnpm add github:` for the framework — exact commands in the README) — no registry overhead, branches and tags work out of the box, internal forks are first-class consumers.

Shared contracts (DORA event, Work ID regex, audit event) live in the monorepo as **mirror modules** — hand-written Zod (TS) and Pydantic (Python), kept in lockstep by contribution rules and a round-trip contract validation job in CI. No code generation: the cost (drift discipline) is lower than the cost of a generator dependency, a build step, and debugging generated code.

`devex check` reads `testCommand` / `lintCommands` from `devex.profile.ts` via a small regex parser (with TS-comment stripping). This is intentionally pragmatic for the PoC — robust enough for the file `devex init` produces, but a hand-edited profile with template-string interpolation or imports for the field values falls back to a warning + explicit `--test` / `--lint` flags. Post-PoC alternatives: emit a sidecar `devex.profile.json`, run a `pnpm devex:checks` helper that evaluates the TS, or call the TypeScript Compiler API from Python via subprocess.

6. Plus criteria delivered

Two of the optional challenge "Plus" criteria are shipped — **Kiro steering files** and **Pre-Push Validation** (`devex check`); details in the README Implementation Status.

> **Appendix — Reference impact on `transactionify`.** Before: 188-line monolithic CDK Stack, no CI/CD, `openapi.yaml` not validated, FinOps tags inline, no DORA. After: ~40-line Stack using `PythonLambdaApi`, generated 5-stage PR pipeline, contract validation enforced, tags fail `cdk synth` if missing, DORA + audit events per stage, multi-env scaffolding (Sandbox/Staging/Prod) ready. Open roadmap items: Integration Pipeline (Sandbox → Staging → Prod promotion), Amazon Q Developer integration, multi-cloud profiles.